Designing & Building an E-Commerce LLD with an AI Coding Agent

A process retrospective — how I plan, design, and debug with AI

> ***A note on format**: I didn't keep a saved chat log for this project, so what follows is not a literal transcript. It's an honest reconstruction of the actual workflow, backed by the real artifacts from the build: my original `lld.drawio` design file (with my own handwritten reasoning notes, quoted verbatim below), the resulting Java source, and the diffs between what I first sketched and what we shipped. Where my first idea was wrong, I've kept it in rather than cleaning it up — that's the part this is meant to show.*

---

1. My process, in one sentence

I design the system before I open the AI tool. The AI is not the architect — I am. It's the implementer who builds against a spec I've already thought through, and the second pair of eyes that catches the parts of my spec that don't hold up once they hit real code.

Concretely, every LLD I build follows the same four phases:

1. **Draw the LLD myself** (draw.io) — actors, entities, relationships, and my first guess at data structures, with my reasoning written directly on the diagram.
2. **Hand the AI agent the diagram + a plain-English walkthrough** of every actor, what they can do, and how entities relate.
3. **Let the agent propose data structures and build the code** against that spec — but treat its first answer as a draft, not the answer.
4. **Debug together**: stress-test edge cases, simplify anything overbuilt, and fix the places where my original diagram was either wrong or under-specified.

The rest of this document walks through that loop on one real project: an e-commerce LLD (`RealWorldLld/ecommerce/`).

---

2. Phase 1 — My own design, before any AI involvement

Before writing a prompt, I drew the actors and entities by hand in draw.io. Three of my own annotations from that file, quoted exactly as I wrote them:

> ***On the Dealer entity:*** `Dealer { dealerId, companyName, products(ArrayList<String>) }`

> ***On data structures generally:*** *"So, Now discuss the data structure, userMap which will be a hashMap have user Information, we can add remove update read from that, Similar we have..."*
>
> ***On products and cart, my first-pass reasoning:*** *"...first user will see, products which is ArrayList I think arrayList would be better for like searching and get as it will take O(n) time, but getting product O(1) I guess (this products will map with dealerId through dealerMap) ... then user do addToCart of product x, set quantity ... so cart is something like FIFO like ordered I want then cart details will sent it to dealer correct?"*

That last note has two mistakes I didn't notice until the build forced me to reckon with them. I'll come back to both.

---

3. Phase 2 — The brief I gave the agent

I didn't ask the agent to "build an e-commerce app." I gave it the diagram above plus a walkthrough in this shape, actor by actor:

Actor	What I told the agent they can do
User	Browse products, add/remove cart items, place orders, view order history
Dealer	Add products, accept/pack/ship orders, manage stock
Delivery Partner	Pick up packages, update delivery status

And the relationships, stated explicitly as foreign-key style links (this is a habit — I never let the agent default to nested object references, because that's how you end up with stale, duplicated state):

```
Product   → belongs to Dealer    (dealerId in Product)
Cart      → belongs to User      (userId in Cart)
CartItem  → belongs to Cart      (part of Cart's item map)
Order     → belongs to User + references Dealer
Delivery  → belongs to Order;  Order references Delivery once shipped
```

I asked the agent to propose the data structure for each collection *based on the query pattern*, not just default to `ArrayList` / `HashMap` out of habit — and then to justify the choice back to me before writing code. That justification step is where most of the real debugging happened, not in the Java syntax.

---

4. Phase 3 — Where my own design was wrong (the debugging that mattered)

Mistake #1: I picked the wrong structure for "products on a dealer," and contradicted myself doing it

My draw.io note said: "*ArrayList would be better for searching and get as it will take O(n) time, but getting product O(1) I guess.*"

Read that again — I argued myself into a contradiction in the same sentence: I said ArrayList lookup is O(n), then immediately guessed O(1) for the same operation. The agent's first question back to me was essentially: "*You need O(1) lookup by product ID — what are you actually optimizing the Dealer's product list for: containment checks, or fetching one product's full data?*"

That question is what exposed the real issue: I'd conflated two different responsibilities in one field.

- **What I had**: `Dealer { products: ArrayList<String> }` — names baked directly onto the dealer, no ID, no O(1) anything.
- **What we shipped**: `Dealer` keeps only `List<Integer> productIds` (just references — `ArrayList` is *correct* here, since a dealer's own product list is short and only ever fully iterated, never looked up by key). The actual `Product` objects live centrally in a `HashMap<Integer, Product>` for O(1) lookup by ID, plus a `TreeMap<Double, List<Integer>>` price index for range queries ("products under ₹2000") and a `HashMap<String, List<Integer>>` category index.

```
// Final Model/Dealer.java — confirms the correction
public class Dealer {
    private int dealerId;
    private String companyName;
    private List<Integer> productIds;   // references only — ArrayList is fine here
    ...
}
```

The lesson I took from this: "pick a fast data structure" is the wrong question. The right question is "fast *for which specific operation*," and a single entity can need different structures for different facets of the same data — which is why product storage ended up split three ways (HashMap for ID lookup, TreeMap for price range, HashMap for category) instead of one structure trying to serve all three.

Mistake #2: I conflated "cart order" with "processing order" — they're not the same FIFO

My note: "*cart is something like FIFO like ordered I want then cart details will sent it to dealer.*"

I was trying to express two different requirements with one structure: 1. The cart should show items in the order I added them. 2. The dealer should process orders in the order they were *received*.

Those sound similar but they're answering different questions on different entities — (1) is about display order on the `Cart`, (2) is about a processing queue on the `Dealer`. Building it forced the split:

- **Cart** → `LinkedHashMap<Integer, CartItem>`. Not a queue at all — I don't need FIFO *eviction* from a cart, I need O(1) lookup by product ID (to update quantity) *and* insertion-order iteration. `LinkedHashMap` gives both; a real FIFO queue would have meant losing O(1) lookup-by-product-id.
- **Dealer order processing** → `ArrayDeque<Integer>` per dealer, used purely as a FIFO queue (`offer` / `poll`), completely separate from the cart.

```
// Final Model/Cart.java
// LinkedHashMap preserves insertion order — cart shows items in the sequence they were a
public class Cart {
    private LinkedHashMap<Integer, CartItem> items;
    ...
}
```

Neither mistake was a typo or a syntax error — both were me describing a real requirement slightly wrong on the diagram, and only catching it because building the thing forced a concrete answer to "what operation does this actually need to support?"

---

5. Phase 4 — What got built once the design was right

With the structures corrected, the rest of the build was implementation against a now-accurate spec. Five patterns came out of asking, for each piece of complexity, "what's actually being managed here":

Complexity in the design	Pattern applied	Why
All 5 services need the same shared state	**Singleton** (`DataStore`)	One in-memory store, not five disconnected copies
`Order` has 6 fields and must be validated before it exists	**Builder** (`OrderBuilder`)	Self-documenting construction + a validation gate at `build()`
User, dealer, *and future channels* all need to react to order status changes	**Observer** (`OrderObserver`)	New notification channels (email, SMS) need zero changes to `OrderService`
Pricing needs to vary (regular, discount, future flash-sale) without an `if/else` ladder	**Strategy** (`PricingStrategy`)	Swappable at runtime on `Cart` , no changes to `CartService`
Placing an order is really 5 coordinated steps across 3 services	**Facade** (`UserService.placeOrder()`)	One call hides cart validation → stock deduction → order creation → notification → cart clear

I didn't go looking for patterns to use — each one came from a specific question in Step 7 of my framework ("is construction multi-step?", "do multiple things need to react to one event?"). The one pattern I considered and *cut*: a `Factory` for `Product` creation. It would have added a layer for a constructor that never varies — three similar lines beat a premature abstraction.

Full reasoning for every data structure (with Big-O comparisons) and the SOLID breakdown is documented in `ecommerce.md` in this same repo.

---

6. One walkthrough — Alice places an order

This is the actual sequence once the design was settled, traced top to bottom:

```
userService.addToCart(aliceId, productId=2, qty=1)
  → DataStore.products.get(2)                          [HashMap O(1)]
  → cart.items.put(2, new CartItem(2, 1, 2499.0))       [LinkedHashMap O(1)]

userService.applyDiscount(aliceId, 10.0)
  → cart.setPricingStrategy(new DiscountPricingStrategy(10.0))   [Strategy swap]
  → cart.calculateTotal() → 2499.0 * 0.9 = 2249.1

userService.placeOrder(aliceId)                         ← Facade entry point
  → CartService.validateCart(aliceId)                  → stock OK
  → deduct stock
  → OrderService.createOrder(...) via OrderBuilder       → validated, built
  → dealerOrderQueues.get(dealerId).offer(orderId)       [ArrayDeque O(1)]
  → userOrderHistory.get(aliceId).offerFirst(orderId)    [ArrayDeque O(1)]
  → notifyObservers(...) → "[USER #1] Order #1 is now: PLACED"
                         → "[DEALER #1] Order #1 updated to: PLACED"

dealerService.shipOrder(...)
  → OrderService.updateStatus(SHIPPED) → notify
  → DeliveryService.createDelivery(orderId) → order.setDeliveryId(...)

deliveryService.updateDeliveryStatus(DELIVERED)
  → mirrors to OrderService.updateStatus(DELIVERED) → notify
```

---

7. What I'd do differently next time

- **Write the contradiction-check into my own process, not just the AI's.** My ArrayList note contradicted itself in one sentence and I didn't catch it until the agent asked a clarifying question. I now re-read my own complexity claims out loud before handing them over.
- **Separate "display order" from "processing order" explicitly on the diagram**, every time a requirement says a collection needs to be "in order" — that word hides two different data structures more often than not.
- **Treat the agent's first data-structure answer as a draft.** The value wasn't that it picked structures faster than I would have — it was that it asked *why* before agreeing with my first guess, which is exactly the role I want it playing.

Source code, the original `lld.drawio` file, and the full pattern/SOLID writeup for this project: `RealWorldLld/ecommerce/` in this repository.